TASK 22: GROUPBY FUNDAMENTALS

Task description: Use Pandas `groupby()` to summarise numerical data across categories and groups.

Task objective: Understand grouping operations essential for exploratory data analysis. Required deliverables are at least five GroupBy examples, examples of sum/mean/count/min/max, and a short interpretation of group-level results.

1. GroupBy: definition and purpose

GroupBy is a data-analysis operation that divides records into groups based on one or more categorical columns, calculates a summary within each group, and returns a compact result. It turns transaction-level data into information that is easier to compare. For example, a retail analyst can calculate total sales by region, average profit by category, or record counts by customer segment.

The process is often called **split–apply–combine**: split rows into groups, apply a calculation to each group, and combine the outputs into a Series or DataFrame. GroupBy does not change the business meaning of the source data; it summarises what is present.

2. Why GroupBy is important

• It reduces large tables to understandable summaries.

• It supports exploratory data analysis (EDA) by revealing differences between categories.

• It compares performance across regions, categories, customer segments, or teams.

• It helps identify patterns that may need further investigation.

• It produces summary tables suitable for reports and dashboards.

• It supports feature engineering, such as category-level averages, when done without data leakage.

3. Tools and dataset

Python runs the program. Pandas provides DataFrames, grouping, and aggregation functions. Jupyter Notebook lets you execute the code step by step and add notes. The included sample dataset is synthetic and Superstore-style. It includes Order_ID, Region, Category, Segment, Sales, Profit, Quantity, and Discount. Its small sample values are for practice only and should not be interpreted as actual retail results.

4. Load and inspect data

```
import pandas as pd\ndf = pd.read_csv('superstore_groupby_sample.csv')\nprint(df.head())\nprint('Shape:', df.shape)\nprint(df.dtyp
```

`read_csv()` reads the CSV file into a DataFrame. `head()` displays the first five rows. `shape` reports the row and column counts. `dtypes` checks whether each column is numeric, text, or another type. Verify that Sales, Profit, Quantity, and Discount are numeric before aggregating.

5. Basic syntax and split–apply–combine

```
df.groupby('Region')['Sales'].sum()
```

The first expression groups the rows by each unique Region. Selecting `Sales` chooses the numeric measure. Calling `sum()` adds the values within each region. The output is a Series indexed by region. If a table is required, `.reset_index(name='Total_Sales')` can turn the index into a column.

6. Aggregation functions in detail

sum() — Adds all non-missing numeric values in a group. It is useful for total sales, profit, and quantity. Confirm whether summing the metric makes business sense.

mean() — Calculates the arithmetic average of non-missing values. It is useful for average sales, profit, or discount. Averages should be interpreted alongside the group size.

count() — Counts non-missing values in a selected column. If the selected field has missing entries, those entries are not included.

size() — Counts rows in each group, regardless of missing values in other columns. This differs from count().

min() — Returns the smallest value in the group, such as the lowest sale.

max() — Returns the largest value in the group, such as the highest sale.

agg() — Applies one or multiple aggregation functions and can assign clear output names.

nunique() — Counts distinct values in a group, excluding missing values by default.

7. Example 1: total sales by region

```
sales_by_region = df.groupby('Region')['Sales'].sum()\nsales_by_region = sales_by_region.sort_values(ascending=False)\nprint(sales_
```

This produces one total for each region. Descending sorting places the largest total first. It helps compare regional contribution. A high total can result from more records, larger transactions, or both.

8. Example 2: average profit by category

```
avg_profit = df.groupby('Category')['Profit'].mean().round(2)\nprint(avg_profit)
```

This calculates the average profit value within each product category. `round(2)` rounds displayed results to two decimal places. A category with the highest average profit might not have the highest total profit; compare totals, averages, and counts.

9. Example 3: count records by segment

```
record_count = df.groupby('Segment')['Order_ID'].count()\nprint(record_count)
```

This counts non-missing Order_ID values per segment. If every row is a unique order and the ID is present, it approximates an order count. If the same order has several line items, this is not a unique-order count. Use `nunique()` for distinct IDs.

10. Example 4: minimum and maximum sales

```
sales_min_max = df.groupby('Category')['Sales'].agg(['min', 'max'])\nprint(sales_min_max)
```

The minimum shows the smallest sale and the maximum shows the largest sale in each category. These extremes describe a range but can be sensitive to outliers. Use average, median, and record count for a fuller picture.

11. Example 5: multiple aggregations by region

```
region_summary = df.groupby('Region').agg(\n    Total_Sales=('Sales', 'sum'),\n    Average_Sales=('Sales', 'mean'),\n    Total_Pro
```

Named aggregation creates a regional scorecard. Each output name is paired with a source column and aggregation function. Multiple metrics are calculated in one operation, which is convenient for reporting.

12. Example 6: grouping by multiple columns

```
region_category_sales = df.groupby(['Region', 'Category'])['Sales'].sum()\nprint(region_category_sales.sort_values(ascending=False
```

This calculates sales for every Region–Category combination. It is more detailed than region-only grouping. The output normally has a MultiIndex, meaning Region and Category form two index levels. Use `.reset_index()` to return them to ordinary columns.

13. Example 7: unique categories by region

```
unique_categories = df.groupby('Region')['Category'].nunique()\nprint(unique_categories)
```

`nunique()` counts distinct category names represented in each region. It does not count orders or sales. It can help compare the breadth of categories across regions.

14. Example 8: several profit statistics by category

```
profit_stats = df.groupby('Category')['Profit'].agg(\n    ['sum', 'mean', 'count', 'min', 'max']\n).round(2)\nprint(profit_stats)
```

The result contains total profit, average profit, non-missing profit count, minimum profit, and maximum profit for each category. Comparing these values prevents overreliance on one metric.

15. Example 9: count rows versus count values

```
row_counts = df.groupby('Region').size()\nvalue_counts = df.groupby('Region')['Order_ID'].count()\nprint(row_counts)\nprint(value_
```

`size()` counts every row in each region. `count()` counts only non-missing Order_ID values. They can differ if Order_ID is missing. In this provided sample IDs are populated, so the counts should match.

16. Example 10: filter groups after aggregation

```
region_summary = df.groupby('Region').agg(\n    Total_Sales=('Sales', 'sum'),\n    Total_Profit=('Profit', 'sum')\n)\nthreshold = 
```

This first summarises sales by region and then filters the summary table to regions whose total sales exceed the median regional total. It is often easier to understand than trying to filter the original rows before the aggregation. Filtering before grouping and

filtering after grouping answer different questions.

17. Interpreting group-level results

• State what is being measured: total sales, average profit, record count, or another statistic.

• Check group sizes before comparing averages.

• Compare total and average measures separately.

• Investigate missing values, repeated IDs, and unusual values.

• Grouping by more columns provides detail but creates more, smaller groups.

• Do not claim a cause based only on a grouped summary.

• Remember this sample is synthetic; findings demonstrate code, not actual retail trends.

18. Common errors and troubleshooting

KeyError: column not found: Print `df.columns` and use the exact column name, including spaces and capitalization.

Sales is stored as object/text: Inspect `df.dtypes`; remove currency symbols or separators carefully, then use `pd.to_numeric(..., errors='coerce')` and inspect newly missing values.

Wrong unique order count: Use `nunique()` on Order_ID if a single order may occupy multiple rows.

Missing groups are absent: Pandas excludes missing group keys by default in many groupby operations. Use `dropna=False` if a missing-key group is needed.

Unexpected output type: A single selected column often returns a Series; use `.reset_index()` or named aggregation for a DataFrame.

Mean appears surprising: Check group count, outliers, and missing data; mean can be affected by extreme values.

19. Save summary files

```
region_summary.to_csv('region_groupby_summary.csv')\nprofit_stats.to_csv('category_profit_summary.csv')\nregion_category_sales.rena
```

The exported CSV summaries can be opened in spreadsheet tools, loaded into dashboards, or used in later analysis. Review the output files to ensure they contain the expected columns and index.

20. Interview questions and answers

Q: What does groupby() do?
Answer: It splits rows into groups based on one or more columns so each group can be summarised independently.

Q: What is aggregation?
Answer: It reduces several values to a summary statistic such as sum, mean, count, min, or max.

Q: How do you group by multiple columns?
Answer: Pass a list, for example `df.groupby(['Region', 'Category'])['Sales'].sum()`.

Q: What is the difference between count() and size()?
Answer: count() counts non-missing values in a selected column; size() counts rows in each group.

Q: How do you count unique order IDs?
Answer: Use `df.groupby('Segment')['Order_ID'].nunique()`.

Q: Why use agg()?
Answer: It calculates multiple statistics in one operation and can provide clear output column names.

Q: Does a higher average mean a higher total?
Answer: No. Average and total are different measures; group size can make their rankings differ.

Q: What is a MultiIndex?
Answer: An index with multiple levels, such as Region and Category after grouping by both columns.